



ResourceHive

One place to discover, share, and book campus resources.

Design, reliability, and user experience.

The problem we solve

Unclear availability. Booking clashes. Scattered coordination.

Members

Find resources, reserve slots, and track bookings.

Administrators

Publish availability, approve access, and manage usage.

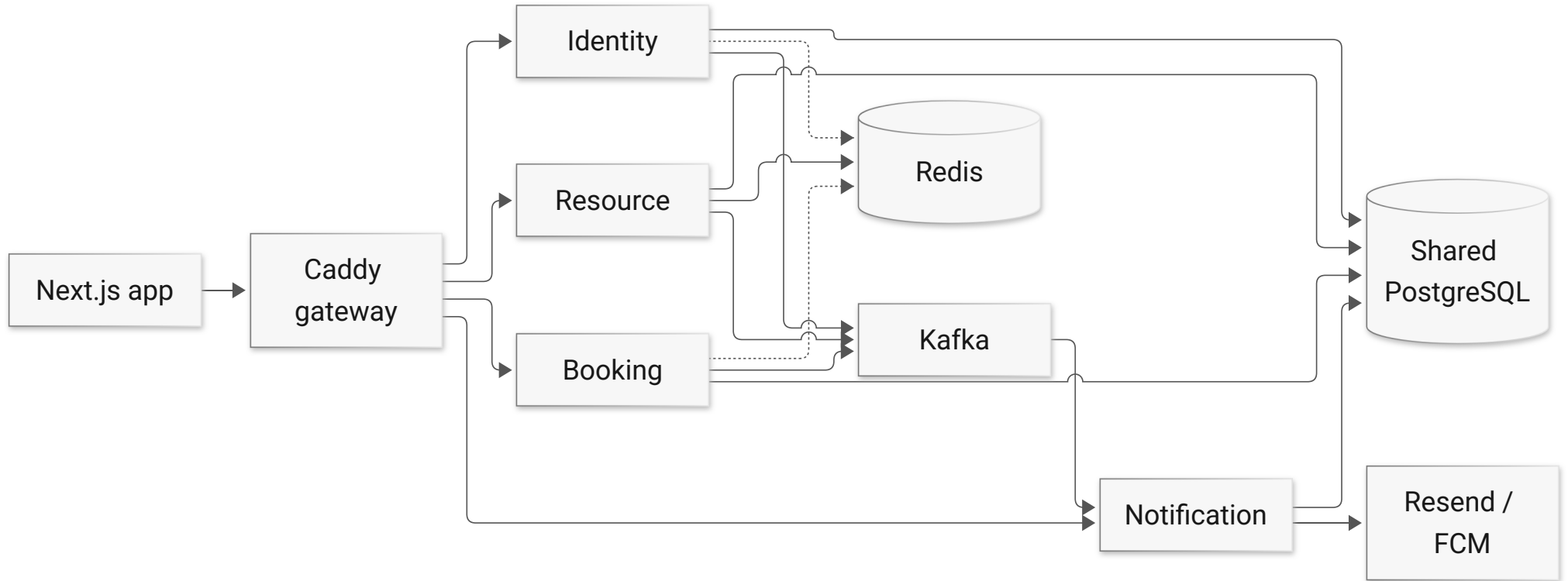
One shared record makes the rules and responsibilities clear.

What we will demonstrate

- | | | |
|---|-------------------------|--|
|  | Find a resource | Catalogue → discover permitted resources |
| <hr/> | | |
|  | Book a slot | Booking dialog → choose available time |
| <hr/> | | |
|  | Check the result | Confirmation + points → verify the reservation |
| <hr/> | | |
|  | Manage usage | Bookings → eligible cancellation or admin completion |

[Open the ResourceHive application ↗](#)

How the system fits together



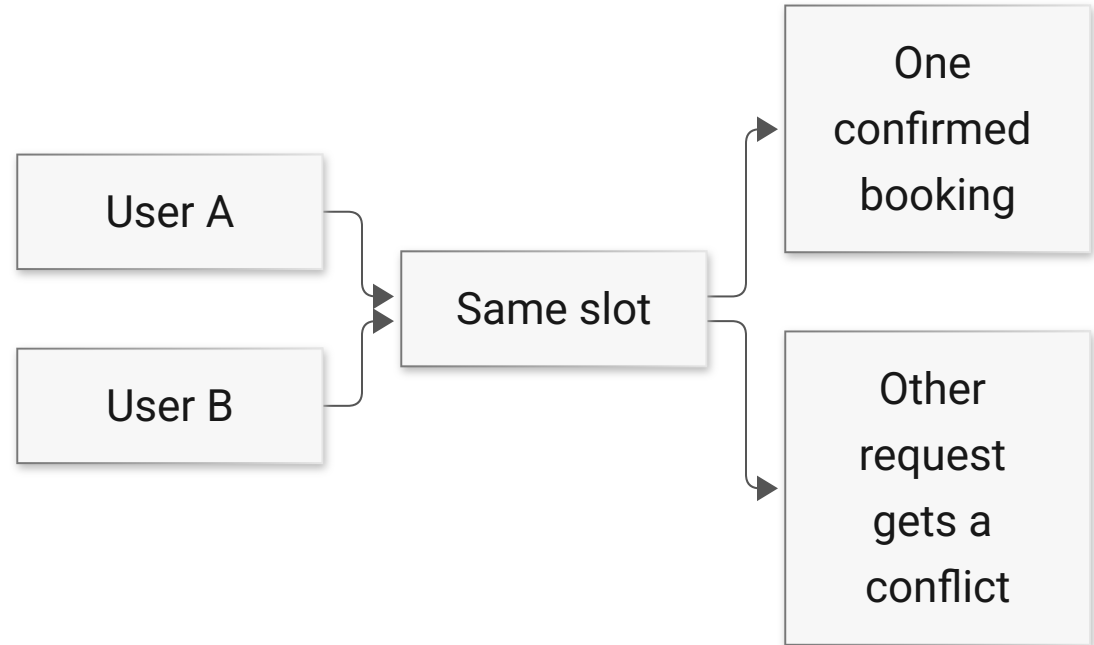
Four NestJS services. One public backend entry point. A shared database.

Preventing double bookings

Two users can select the same available slot.

How it works

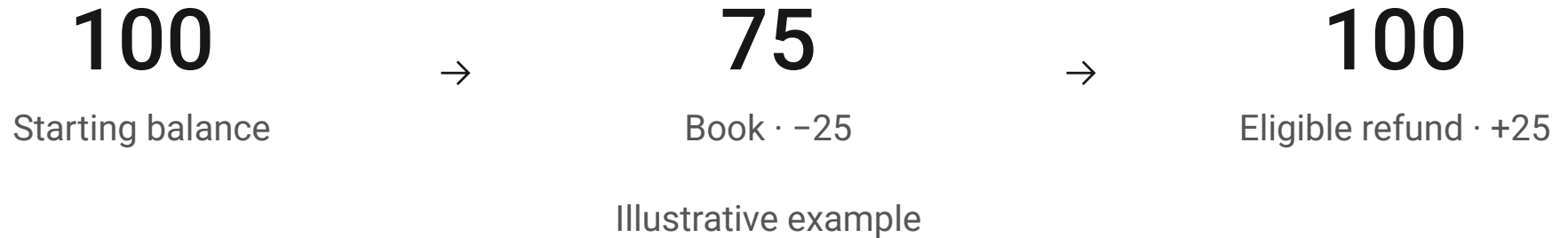
Serializable transactions.
Up to three attempts.
Database constraints.



| **Benefit:** the booking and points deduction succeed together.

Keeping points accurate and traceable

Users need a correct balance and an explainable history.



Append-only history

Preserve each movement.

Maintained balance

Read the current total.

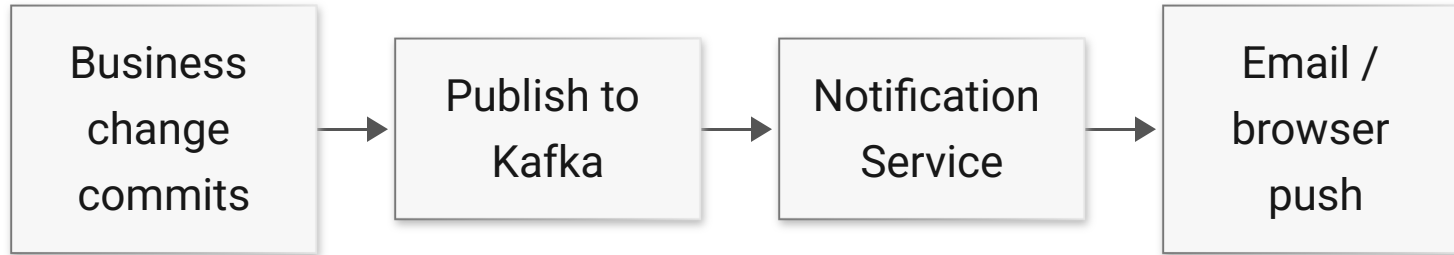
Unique entries

Prevent duplicate charges.

| **Benefit:** traceable changes with an efficient balance lookup.

Processing notifications asynchronously

Email and push delivery can be slow or unavailable.

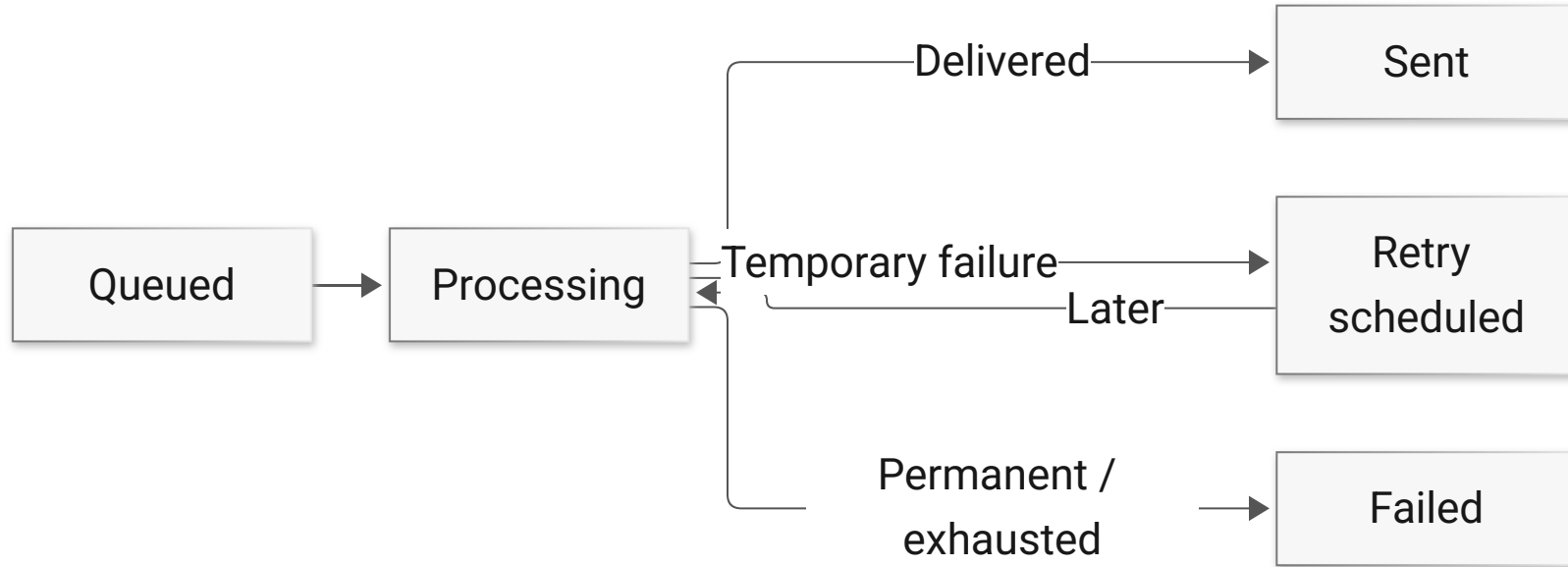


How it works: queue delivery work after the business change.

| **Benefit:** the business request does not wait for provider delivery.

Recovering from notification failures

Messages can repeat. Delivery providers can fail.



How it works: deduplicate events and store delivery progress.

Invalid commands → dead-letter topic, separate from delivery retries.

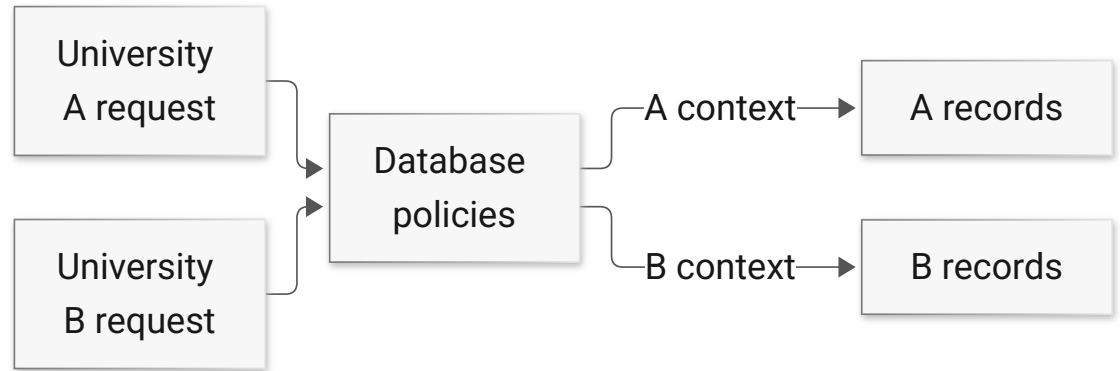
| Benefit: controlled retries and recovery after interruptions.

Keeping university data separate

One platform serves multiple universities.

How it works

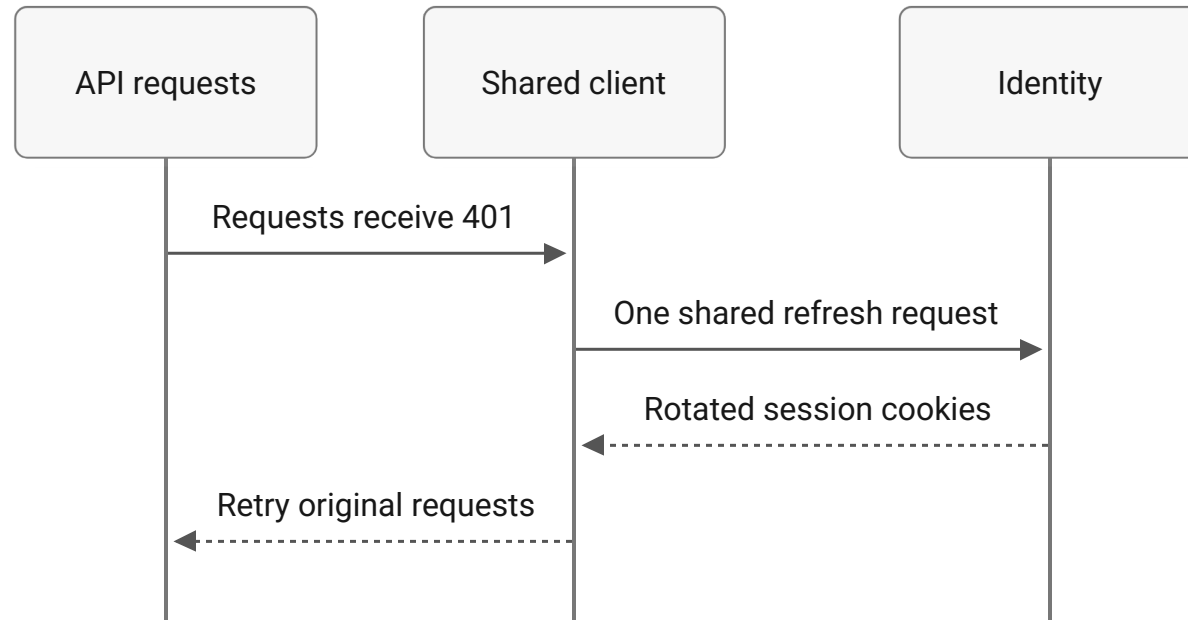
Application access checks.
Database Row-Level Security.
Context set per transaction.



| **Benefit:** another enforcement layer for university boundaries.

Keeping users signed in smoothly

Several requests can discover an expired session together.

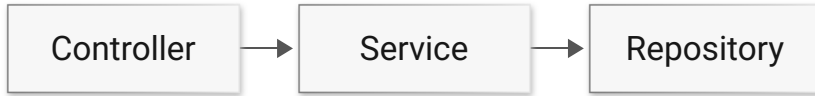


| **Benefit:** fewer login interruptions and competing refresh requests.

Other techniques that improve the experience

	Redis caching	Reduce repeated database reads.
	Live updates	Refresh relevant views through Socket.IO events.
	Controlled loading	Page, parallelize, and cancel requests.
	Clear UI states	Explain waiting, errors, and booking conflicts.
	Gateway + health checks	Compress responses and coordinate startup.

How the code follows the design



Shared packages

Authentication

Database access

Notification contracts

Real excerpt · BookingService.createBooking

```
const booking = await this.prisma.$transaction(  
  (transaction) => this.createWithinTransaction(  
    resourceSlotId, user, transaction,  
  ),  
  {  
    isolationLevel:  
      Prisma.TransactionIsolationLevel.Serializable,  
    maxWait: 10000,  
    timeout: 30000,  
  },  
);
```

TypeScript types · Input validation · Consistent errors · Automated checks

How we test the system

Test layer	Concrete example
Unit / component	Booking rules and form states
API	Login, permissions, and response errors
Database integration	Booking and points commit together
Concurrency	Competing bookings; repeated events
Regression	Rerun existing suites in CI

Mocks check isolated behavior. Real-database tests check integration and constraints.

Testing evidence and its limits

Recorded checks

27 September 2026 · bcc0bd9

Recorded checks	Result
Frontend	114 tests passed
Booking unit tests	43 tests passed
Booking integration / concurrency	4 / 1 passed
Manual journeys and provider flows	Partially verified

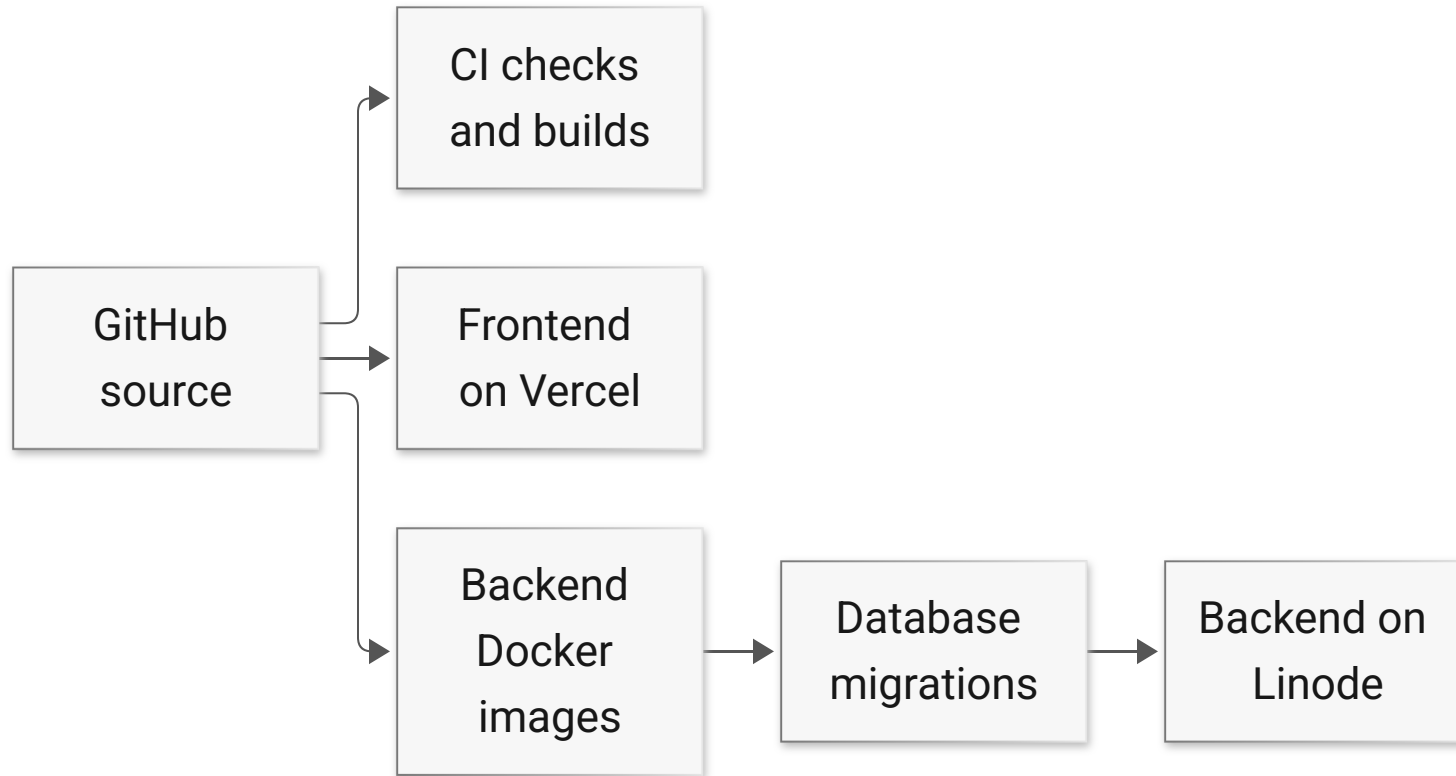
Local load results show lower success as concurrency increases.

Local load testing

Project-reported success rates

Virtual users	Success rate
Up to 20	Above 90%
30	75%
50	49%

From source code to a running application



Versioned code. Repeatable builds. Committed migrations.

CI and deployment are separate workflows. Demo the deployed application in a browser.

Tradeoffs and next improvements



Shared database

Simplifies consistency; creates coupling between services.



Commit-to-event gap

An outbox would recover events after publishing failures.



Cache and realtime hardening

Improve invalidation and authorize room subscriptions.

Questions

[Open ResourceHive ↗](#)